```
INFO: Creating docker container
INFO: Copy goss files into container
Successfully copied 13.7MB to zookeeper:/goss
INFO: Container name: zookeeper
INFO: Sleeping for 5
INFO: Container health
INFO: Running Tests
1..3
ok 1 - Process: java: running: matches expectation: true
not ok 2 - Port: tcp6:9092: listening: Expected false to equal true
ok 3 - Port: tcp6:2181: listening: matches expectation: true
INFO: Stopping container
INFO: Test ran for a total of 8 seconds
```

Figure 7: Running dcgoss ZooKeeper tests

```
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092
     - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTE
XT://127.0.0.1:9092
      - KAFKA_CFG_ZOOKEEPER_CONNECT=zookeeper:2181
      - ALLOW_PLAINTEXT_LISTENER=yes
    depends_on:
      - zookeeper

--- kafka.yaml ---

port:
  tcp6:2181:
    listening: true
  tcp6:9092:
    listening: true

process:
  java:
    running: true
```

It's time to fire dcgoss to test each service container. Run the command *docker run -it --rm -v ${PWD}/:/etc/goss:ro -v /var/run/docker.sock:/var/run/docker.sock:ro -e GOSS_FILE=kafka.yaml -e GOSS_SLEEP=5 -e GOSS_FILES_STRATEGY=cp -e GOSS_OPTS='--color -format tap' dcgoss:0.4.9 dcgoss run zookeeper* to run all goss tests in the ZooKeeper service container. You should see all the ZooKeeper tests passed, as shown in Figure 7.

Run the command *docker run -it --rm -v ${PWD}/:/etc/goss:ro -v /var/run/docker.sock:/var/run/docker.sock:ro -e GOSS_FILE=kafka.yaml -e GOSS_SLEEP=20 -e GOSS_FILES_STRATEGY=cp -e GOSS_OPTS='--color -format tap' dcgoss:0.4.9 dcgoss run kafka* to goss acceptance test the Kafka container. You should see all the Kafka tests passing now.

We have combined both ZooKeeper and Kafka goss tests in a common file for quick testing showing 'ok', 'not ok' and 'skip test' results. But you can separate the tests in their respective goss files and mention the corresponding one through the *GOSS_FILE* environment variable.

Finally, we can clean up the running service containers brought up by dcgoss using the command *docker ps -q|xargs -I % docker rm -f %* to say goodbye to it.

Last but not the least, Kubernetes has become a *de-facto* platform to run and manage containerised services at scale. The goss project also provides another wrapper in the form of kgoss to acceptance test containers running in Kubernetes pods. You need to run kgoss on a machine where kubectl is already installed and configured to talk to your local or remote Kubernetes cluster. The kgoss help page describes its installation and usage clearly.

Container Structure Tests should be the first stage in your container acceptance testing pipeline to static test container images. The goss wrappers created for services running in containers are very useful for runtime quality gating mechanisms in modern build and release pipelines. This way we can be highly confident that the created container images pass all the necessary acceptance criteria. END

References

- Container Structure Tests GitHub project, *https://github.com/GoogleContainerTools/container-structure-test*
- goss GitHub project, *https://github.com/goss-org/goss*
- dgoss documentation, *https://github.com/goss-org/goss/tree/master/extras/dgoss*
- dcgoss documentation, *https://github.com/goss-org/goss/tree/master/extras/dcgoss*
- kgoss documentation, *https://github.com/goss-org/goss/tree/master/extras/kgoss*

By: Ankur Kumar

The author is a passionate hacker/researcher of free and open source software. He loves to explore cutting-edge technologies, ancient sciences, quantum spirituality, various genres of music, mystical literature and art.